Übungsblatt 6

Asynchrones JavaScript, Browser-Speicher & Web-APIs

5430UE Web and Data Engineering

20. Mai 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Grundlegendes Verständnis von asynchronem JavaScript.
- Einblick in die Verarbeitung externer Daten über die Fetch API.
- Grundlegendes Verständnis von Web Workers und clientseitiger Speicherung.

Währungsrechner - Fetch API und asynchrones JavaScript

Währungsrechner - Fetch API und asynchrones JavaScript

Implementieren Sie einen Währungsrechner, der Beträge von Euro (EUR) in US-Dollar (USD) oder Schweizer Franken (CHF) umrechnet. Das Projekt *waehrungsrechner* ist in Stud.IP hinterlegt. Die Dateien *index.html* und *style.css* sind bereits vollständig funktionsfähig und müssen nicht verändert werden.

Vervollständigen Sie die *script.js* an den markierten Stellen.

- Bauen Sie die URL dynamisch mit den Parametern `amount`, `from=EUR` und `to` (Zielwährung) zusammen.
- Wandeln Sie die API-Antwort in JSON um und extrahieren Sie den umgerechneten Wert aus dem Feld `data.rates`.
- Genauere Informationen zur Form der URL und der Struktur des Responses finden Sie unter Frankfurter API v1
- Geben Sie das Ergebnis formatiert (2 Nachkommastellen) im Element `#result` aus.
- Registrieren Sie die notwendigen EventListener, damit sich das Ergebnis bei jeder Eingabe oder Auswahl automatisch aktualisiert.

Währungsrechner - Fetch API und asynchrones JavaScript — Lösung (1/3)

script.js

```
const amountEl = document.getElementById('amount');
const currencyEl = document.getElementById('targetCurrency');
const resultEl = document.getElementById('result');

// --- Luecke ergaenzen ---
async function convert() {
  const amount = amountEl.value;
  const target = currencyEl.value;

  if (!amount || amount <= 0) {
    resultEl.innerText = "Bitte Betrag eingeben";
    return;
  }
```

Währungsrechner - Fetch API und asynchrones JavaScript — Lösung (2/3)

```
  // --- START: Hier ergaenzen (Loesung) ---
  try {
    const url = `https://api.frankfurter.dev/v1/latest?amount=${amount}&from=EUR&to=${target}`;

    const response = await fetch(url);

    if (!response.ok) throw new Error("API Fehler");

    const data = await response.json();

    // Das Ergebnis steht in data.rates[target], siehe API Doku
    const convertedValue = data.rates[target];

    resultEl.innerHTML = `${amount} EUR = ${convertedValue.toFixed(2)} ${target}`;
  } catch (error) {
    resultEl.innerText = "Fehler beim Laden der Daten.";
  }
  // --- ENDE ---
}
```

Währungsrechner - Fetch API und asynchrones JavaScript — Lösung (2/3)

```
// --- START: ---
// Event-Listener fuer automatische Aktualisierung
amountEl.addEventListener('input', convert);
currencyEl.addEventListener('change', convert);
// --- ENDE ---
```

Produkte laden - Fetch API und asynchrones JavaScript

Produkte laden - Fetch API und asynchrones JavaScript

Implementieren Sie die Funktion `loadProduct(id)`, um Produktdetails von einer externen API abzurufen und anzuzeigen.

- *Abruf*: Rufen Sie mittels fetch die URL `https://api.example.com/products/{id}` auf (GET-Request).
- *Validierung*: Prüfen Sie `response.ok`. Werfen Sie bei Fehlern (Netzwerk oder HTTP-Status >= 400) eine Exception.
- *Rendering*: Wandeln Sie das Ergebnis in JSON um und geben Sie die Felder `name`, `price` und `description` im Element `<div id="product-detail">` aus.
- *Fehlerbehandlung*: Fangen Sie alle Fehler mit `try...catch` ab und zeigen Sie eine entsprechende Fehlermeldung im selben `<div>` an.
- Verwenden Sie konsequent die `async/await`-Syntax.

Hinweis: Die gegebene API-Adresse stellt lediglich eine Dummy-Adresse dar und liefert kein Ergebnis.

Produkte laden - Fetch API und asynchrones JavaScript — Lösung (1/2)

```
async function loadProduct(id) {
  const detailDiv = document.getElementById('product-detail');

  try {
    // 1. API aufrufen (GET ist Standard)
    const response = await fetch(`https://api.example.com/products/${id}`);

    // 2. Fehlerpruefung (Netzwerkfehler vs. HTTP-Status >= 400)
    if (!response.ok) {
      throw new Error(`HTTP-Fehler! Status: ${response.status}`);
    }
```

Produkte laden - Fetch API und asynchrones JavaScript — Lösung (2/2)

```
    // 3. JSON umwandeln und rendern
    const product = await response.json();
    detailDiv.innerHTML = `
    <h2>${product.name}</h2>
    <p>Preis: ${product.price}</p>
    <p>${product.description}</p>
    `;

  } catch (error) {
    // 4. Fehleranzeige im selben div
    detailDiv.textContent = "Fehler beim Laden des Produkts.";
  }
}
```

Anmerkungen:

- `response.ok` prüft HTTP-Status 200–299.
- `fetch` wirft bei Netzwerkfehlern eine Exception, aber **nicht** bei HTTP 4xx/5xx — daher die manuelle `if (!response.ok)` Prüfung.

Web Workers und Local Storage

Web Workers und Local Storage

1. **Web Workers:**
    1. Erklären sie kurz, was man unter einem *Web Worker* versteht.
    2. Ein Online-Shop berechnet Produktempfehlungen per komplexem Scoring-Algorithmus. Nach der Implementierung friert die UI kurz ein. Erklären Sie: Warum friert die UI ein? Welche Technologie löst dieses Problem?
2. **Szenario:**

    Ein Nutzer füllt ein mehrstufiges Registrierungsformular aus. Die bereits eingegebenen Daten sollen bei einem versehentlichen Neuladen der Seite erhalten bleiben, nach dem Schließen des Tabs jedoch verworfen werden. Zusätzlich soll die vom Nutzer gewählte Spracheinstellung auch nach einem Browser-Neustart erhalten bleiben.

    Welche Möglichkeiten bietet Web Storage zur clientseitigen Persistenz? Vergleichen Sie zwei davon hinsichtlich:
    - Lebensdauer (bis wann gehen Daten verloren?)
    - Speicherkapazität (ungefähr)
    - Typischer Einsatz

Web Workers und Local Storage — Lösung (1/2)

Zu 1) Web Workers:

1. Ein Web Worker ist eine Technologie, mit der JavaScript-Code in einem separaten Hintergrund-Thread ausgeführt werden kann, ohne den Main Thread und damit die Benutzeroberfläche zu blockieren.
2.

Web Workers und Local Storage — Lösung (1/2)

Zu 1) Web Workers:

1. Ein Web Worker ist eine Technologie, mit der JavaScript-Code in einem separaten Hintergrund-Thread ausgeführt werden kann, ohne den Main Thread und damit die Benutzeroberfläche zu blockieren.

2. **Ursache**:
   JavaScript im Browser läuft standardmäßig single-threaded im Main Thread. Führt dieser aufwendige synchrone Berechnungen aus, können währenddessen keine UI-Updates oder Benutzerinteraktionen (z.B. Klicks oder Scrollen) verarbeitet werden. Dadurch wirkt die Oberfläche eingefroren.

Web Workers und Local Storage — Lösung (1/2)

Zu 1) Web Workers:

1. Ein Web Worker ist eine Technologie, mit der JavaScript-Code in einem separaten Hintergrund-Thread ausgeführt werden kann, ohne den Main Thread und damit die Benutzeroberfläche zu blockieren.

2. **Ursache**:
   JavaScript im Browser läuft standardmäßig single-threaded im Main Thread. Führt dieser aufwendige synchrone Berechnungen aus, können währenddessen keine UI-Updates oder Benutzerinteraktionen (z.B. Klicks oder Scrollen) verarbeitet werden. Dadurch wirkt die Oberfläche eingefroren.

   Lösung:
   Der Einsatz von Web Workers. Diese lagern rechenintensive Aufgaben in einen separaten -Hintergrund-Thread aus, sodass der Main Thread für die Interaktion mit dem Nutzer frei/responsive bleibt. Die Kommunikation erfolgt asynchron, z.,B. über `postMessage`.

Web Workers und Local Storage — Lösung (2/2)

Zu 2) Lokale Speicherung:

Im Web Storage stehen insbesondere **sessionStorage** und **localStorage** zur Verfügung. Für das gegebene Szenario eignet sich:

- **sessionStorage** für die temporäre Speicherung der Formulardaten
- **localStorage** für die dauerhafte Speicherung der Spracheinstellung

sessionStorage	localStorage
Lebensdauer	
Kapazität	
Typischer Einsatz	

Web Workers und Local Storage — Lösung (2/2)

Zu 2) Lokale Speicherung:

Im Web Storage stehen insbesondere **sessionStorage** und **localStorage** zur Verfügung. Für das gegebene Szenario eignet sich:

- **sessionStorage** für die temporäre Speicherung der Formulardaten
- **localStorage** für die dauerhafte Speicherung der Spracheinstellung

	sessionStorage	localStorage
Lebensdauer	Bis der Tab/Fenster geschlossen wird → Daten weg	
Kapazität		
Typischer Einsatz		

Web Workers und Local Storage — Lösung (2/2)

Zu 2) Lokale Speicherung:

Im Web Storage stehen insbesondere **sessionStorage** und **localStorage** zur Verfügung. Für das gegebene Szenario eignet sich:

- **sessionStorage** für die temporäre Speicherung der Formulardaten
- **localStorage** für die dauerhafte Speicherung der Spracheinstellung

	sessionStorage	localStorage
Lebensdauer	Bis der Tab/Fenster geschlossen wird → Daten weg	Dauerhaft bis zur expliziten Löschung (auch nach Browser-Neustart)
Kapazität		
Typischer Einsatz		

Web Workers und Local Storage — Lösung (2/2)

Zu 2) Lokale Speicherung:

Im Web Storage stehen insbesondere **sessionStorage** und **localStorage** zur Verfügung. Für das gegebene Szenario eignet sich:

- **sessionStorage** für die temporäre Speicherung der Formulardaten
- **localStorage** für die dauerhafte Speicherung der Spracheinstellung

	sessionStorage	localStorage
Lebensdauer	Bis der Tab/Fenster geschlossen wird → Daten weg	Dauerhaft bis zur expliziten Löschung (auch nach Browser-Neustart)
Kapazität	ca. 5 MB	
Typischer Einsatz		

Web Workers und Local Storage — Lösung (2/2)

Zu 2) Lokale Speicherung:

Im Web Storage stehen insbesondere **sessionStorage** und **localStorage** zur Verfügung. Für das gegebene Szenario eignet sich:

- **sessionStorage** für die temporäre Speicherung der Formulardaten
- **localStorage** für die dauerhafte Speicherung der Spracheinstellung

	sessionStorage	localStorage
Lebensdauer	Bis der Tab/Fenster geschlossen wird → Daten weg	Dauerhaft bis zur expliziten Löschung (auch nach Browser-Neustart)
Kapazität	ca. 5 MB	ca. 5–10 MB
Typischer Einsatz		

Web Workers und Local Storage — Lösung (2/2)

Zu 2) Lokale Speicherung:

Im Web Storage stehen insbesondere **sessionStorage** und **localStorage** zur Verfügung. Für das gegebene Szenario eignet sich:

- **sessionStorage** für die temporäre Speicherung der Formulardaten
- **localStorage** für die dauerhafte Speicherung der Spracheinstellung

	sessionStorage	localStorage
Lebensdauer	Bis der Tab/Fenster geschlossen wird → Daten weg	Dauerhaft bis zur expliziten Löschung (auch nach Browser-Neustart)
Kapazität	ca. 5 MB	ca. 5–10 MB
Typischer Einsatz	Temporäre Formularzustände, Zwischenspeicherung pro Sitzung	

Web Workers und Local Storage — Lösung (2/2)

Zu 2) Lokale Speicherung:

Im Web Storage stehen insbesondere **sessionStorage** und **localStorage** zur Verfügung. Für das gegebene Szenario eignet sich:

- **sessionStorage** für die temporäre Speicherung der Formulardaten
- **localStorage** für die dauerhafte Speicherung der Spracheinstellung

	sessionStorage	localStorage
Lebensdauer	Bis der Tab/Fenster geschlossen wird → Daten weg	Dauerhaft bis zur expliziten Löschung (auch nach Browser-Neustart)
Kapazität	ca. 5 MB	ca. 5–10 MB
Typischer Einsatz	Temporäre Formularzustände, Zwischenspeicherung pro Sitzung	Nutzereinstellungen, dauerhafte Präferenzen, Warenkorb-Entwurf

Hinweis zu den Übungssitzungen am 20.05.2026: Projektauftakt

Hinweis zu den Übungssitzungen am 20.05.2026: Projektauftakt

In den Sitzungen am 20.05.2026 werden neben der Besprechung der Übungsaufgaben von Blatt 06 auch die Projektaufgabe vorgestellt sowie organisatorische Hinweise zum Projekt besprochen. Die Teilnahme an einer dieser Sitzungen ist verpflichtende Voraussetzung für die Teilnahme am Projekt.

Hinweis: Die Übungsgruppe von 10–12 Uhr war bislang durchgehend gut besucht, sodass zu erwarten ist, dass auch nächste Woche nur noch wenige Sitzplätze frei sind. Teilnehmende, die bisher noch nicht an den Übungen teilgenommen haben, werden daher gebeten, nach Möglichkeit bevorzugt die Sitzung von 8–10 Uhr zu besuchen.

Materialien

Materialien zum Übungsblatt

- Aufgabe *Währungsrechner - Fetch API und asynchrones JavaScript*: waehrungsrechner.zip

Lösungen:

- Aufgabe *Währungsrechner - Fetch API und asynchrones JavaScript*: waehrungsrechner_lsg.zip